Memoria técnica: SmartCampus con Docker Compose y Kubernetes

Datos de la práctica

Dato	Valor
Asignatura	Despliegue y monitorización en la nube
Práctica	Despliegue de una aplicación con Docker y Kubernetes (SmartCampus)
Integrantes del grupo	Rodrigo Caminero y Juan Romero
Fecha de entrega	2 de noviembre de 2026
Repositorio	github.com/Rodrcam/smartcampus

1. Introducción

SmartCampus Solutions necesita un prototipo de su plataforma de gestión de aulas y reservas que se pueda desplegar con contenedores, tanto con Docker Compose como con Kubernetes. Hemos desarrollado una aplicación de tres servicios y la hemos desplegado de las dos formas, con el mismo código y las mismas imágenes, para comparar ambos enfoques con pruebas propias.

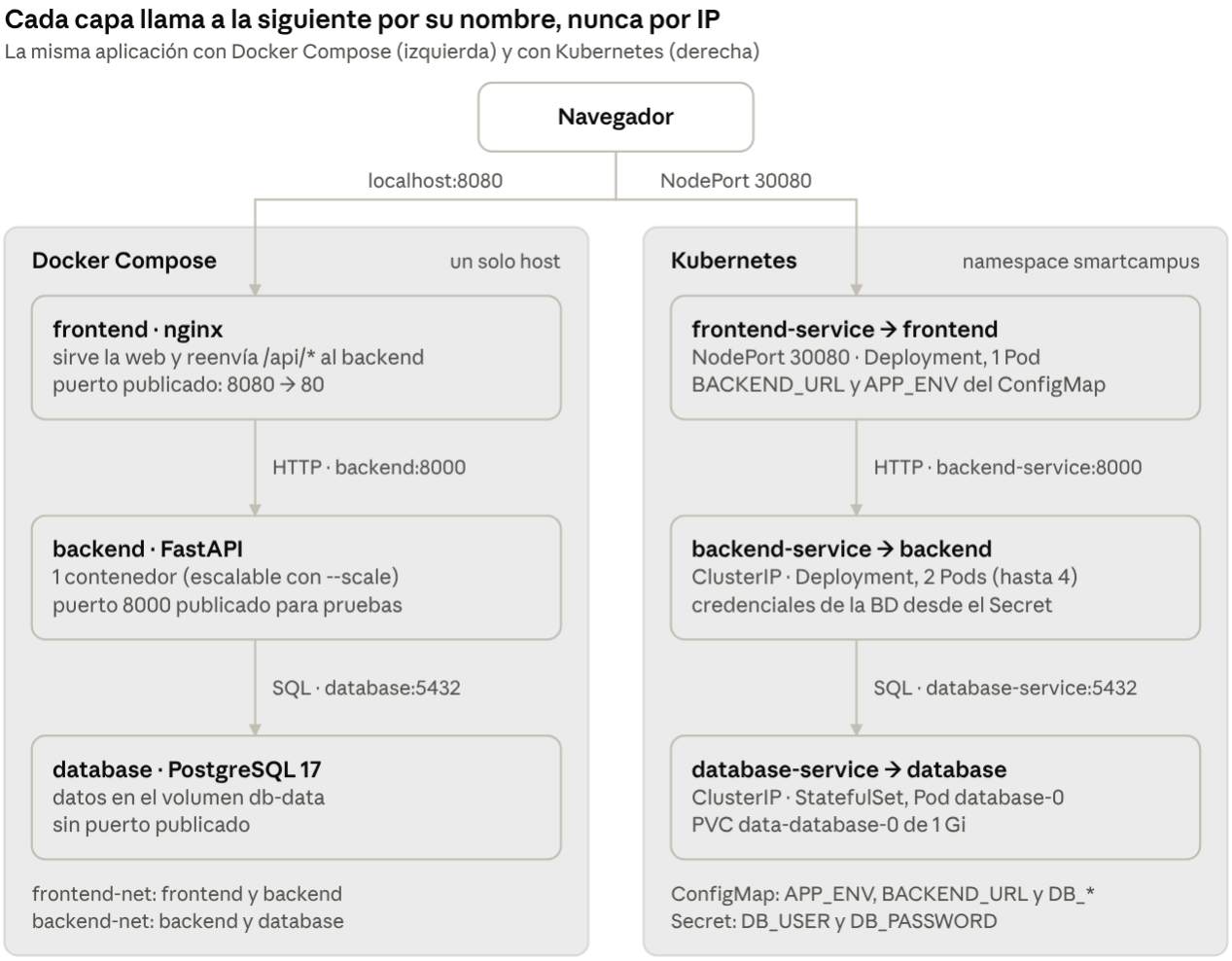
La aplicación permite consultar, crear y eliminar **aulas** (id, nombre, edificio, capacidad y equipamiento) y **reservas** (id, aula, usuario, fecha, hora de inicio y hora de fin). Además, impide guardar dos reservas solapadas de la misma aula.

Componente	Tecnología	Imagen Docker
Frontend	HTML y JavaScript sin frameworks, servidos por nginx 1.29	`smartcampus-frontend:1.0`, construida sobre `nginx:1.29-alpine`
Backend	API REST con Python 3.12, FastAPI y psycopg 3	`smartcampus-backend:1.0`, construida sobre `python:3.12-slim`
Base de datos	PostgreSQL 17	`postgres:17-alpine` (imagen oficial)

El backend ofrece los endpoints mínimos del enunciado (`GET /health` , `GET /rooms` , `GET /rooms/{id}` y `GET /bookings`) y también los opcionales (`POST /rooms` , `POST /bookings` , `DELETE /rooms/{id}` y `DELETE /bookings/{id}`). El código, los Dockerfiles, el `docker-compose.yml` , los manifiestos de Kubernetes y la guía de pruebas están en el repositorio indicado en la portada.

2. Arquitectura

La aplicación tiene tres capas en cadena: el navegador solo habla con el frontend, el frontend con el backend y el backend con la base de datos. La arquitectura es la misma en los dos despliegues; cambian los nombres y quién mantiene los servicios en marcha.



Arquitectura de SmartCampus en Docker Compose y en Kubernetes

En Compose cada capa llega a la siguiente por el nombre del servicio, y en Kubernetes por el nombre de su Service. La base de datos nunca se expone fuera, y en Compose el frontend ni siquiera comparte red con ella.

3. Despliegue con Docker

Cada servicio corre en su propio contenedor: dos imágenes las construimos con nuestros Dockerfiles y la base de datos usa la imagen oficial de PostgreSQL. Un único `docker-compose.yml` levanta los tres con `docker compose up`.

3.1 Dockerfiles

El Dockerfile del backend:

```
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app ./app

RUN useradd --uid 1000 --no-create-home appuser
USER appuser

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

- **Imagen base `slim`**: trae Python oficial sin compiladores ni herramientas que la aplicación no usa, así que la imagen es más pequeña y tiene menos superficie de ataque.
- **Dependencias antes que el código**: Docker guarda cada instrucción como una capa. Si solo cambia el código, reutiliza la capa del `pip install` y la reconstrucción tarda segundos.
- **`PYTHONUNBUFFERED=1`**: los logs salen al momento en `docker logs` y `kubectl logs`.
- **Usuario sin privilegios**: la API no se ejecuta como root dentro del contenedor.
- **`CMD` en formato exec**: uvicorn es el proceso 1 del contenedor y recibe directamente la señal de parada, así que termina de forma ordenada.
- **`.dockerignore`**: excluye cachés de Python del contexto de construcción.

El frontend parte de `nginx:1.29-alpine` y solo copia los ficheros estáticos y una plantilla de configuración. Al arrancar, la imagen oficial de nginx sustituye en la plantilla las variables `${BACKEND_URL}` y `${APP_ENV}` por sus valores. Gracias a eso, la misma imagen sirve en Compose y en Kubernetes: solo cambian las variables de entorno.

```
resolver ${NGINX_LOCAL_RESOLVERS} valid=10s ipv6=off;

location /api/ {
    set $backend ${BACKEND_URL};
    rewrite ^/api/(.*)$ /$1 break;
    proxy_pass $backend;
}
```

3.2 Imágenes utilizadas

Imagen	Origen	Uso
`smartcampus-backend:1.0`	Construida con `backend/Dockerfile`	API REST
`smartcampus-frontend:1.0`	Construida con `frontend/Dockerfile`	Web y proxy inverso hacia el backend
`postgres:17-alpine`	Imagen oficial de Docker Hub	Base de datos

Las dos imágenes propias llevan una etiqueta de versión (`1.0`) y no `latest` . Así sabemos qué versión se ejecuta y Kubernetes no intenta descargarlas de un registro (sección 4.1).

3.3 Docker Compose

Aspecto	Configuración	Motivo
Servicios	`frontend` , `backend` y `database`	El nombre de cada servicio es también su nombre DNS dentro de la red de Compose
Redes	`frontend-net` (frontend y backend) y `backend-net` (backend y base de datos)	El frontend no puede llegar a la base de datos: si alguien compromete nginx, PostgreSQL no queda a su alcance

Aspecto	Configuración	Motivo
Puertos	`8080:80` en el frontend y `8000:8000` en el backend; PostgreSQL no publica ninguno	Solo se expone lo que necesita el usuario. El 8000 queda publicado para probar la API directamente
Variables de entorno	`APP_ENV`, `BACKEND_URL`, `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER` y `DB_PASSWORD`, con valores por defecto que se cambian en un fichero `.env`	La configuración no está dentro de las imágenes, así que la misma imagen sirve en cualquier entorno
Dependencias	`depends_on` con `condition: service_healthy` y healthchecks (`pg_isready` en PostgreSQL y `/health/live` en el backend)	Un `depends_on` simple solo espera a que el contenedor arranque, no a que PostgreSQL acepte conexiones
Volúmenes	Volumen con nombre `db-data` en `/var/lib/postgresql/data` y `database/init.sql` montado en `/docker-entrypoint-initdb.d`	Los datos sobreviven a `docker compose down`, y el script crea las tablas y los datos de ejemplo la primera vez
Configuración de la base de datos	`POSTGRES_DB`, `POSTGRES_USER` y `POSTGRES_PASSWORD` a partir de las mismas variables que usa el backend	Backend y base de datos comparten las credenciales sin escribirlas dos veces
Reinicio	`restart: unless-stopped` en los tres servicios	Si el proceso de un contenedor termina por sí solo, Docker lo vuelve a arrancar (Experimento 2)

3.4 Comunicación entre servicios

El navegador solo habla con nginx. Las páginas llaman a rutas `/api/...` del mismo origen, y nginx las reenvía a `http://backend:8000` quitando el prefijo `/api`. El backend se conecta a PostgreSQL con `DB_HOST=database`. Los dos nombres los resuelve el DNS interno de Docker (`127.0.0.11`), que solo existe en las redes definidas por el usuario.

Configuramos nginx para que resuelva el nombre del backend en cada petición (con 10 segundos de caché) y no solo al arrancar. Si el contenedor del backend se recrea con otra IP, nginx la encuentra sin reiniciarse. Y si hay varias réplicas, el DNS devuelve todas sus IPs y nginx las alterna.

4. Despliegue con Kubernetes

Cada servicio de Compose se traduce en un controlador que crea los Pods y en un Service que les da un nombre estable, todo dentro del namespace `smartcampus`. Usamos Minikube y las mismas imágenes, que se cargan en el clúster con `minikube image load`. Los manifiestos están en `k8s/` y van numerados porque `kubectl apply -f .` los aplica en orden alfabético, y el namespace tiene que existir antes que el resto.

Fichero	Recurso	Nombre	Función
`00-namespace.yaml`	Namespace	`smartcampus`	Agrupa todos los recursos de la aplicación
`01-configmap.yaml`	ConfigMap	`smartcampus-config`	`APP_ENV`, `BACKEND_URL`, `DB_HOST`, `DB_PORT` y `DB_NAME`
`02-secret.yaml`	Secret	`smartcampus-db-secret`	`DB_USER` y `DB_PASSWORD`
`03-database-init-configmap.yaml`	ConfigMap	`database-init`	El script `init.sql`, montado como fichero en PostgreSQL
`10-database-statefulset.yaml`	StatefulSet	`database`	PostgreSQL con un volumen propio de 1 Gi
`11-database-service.yaml`	Service (ClusterIP y headless)	`database-service` y `database-headless`	Acceso estable a PostgreSQL e identidad de red del StatefulSet
`20-backend-deployment.yaml`	Deployment	`backend`	API REST con 2 réplicas
`21-backend-service.yaml`	Service (ClusterIP)	`backend-service`	Reparte las peticiones entre las réplicas del backend
`30-frontend-deployment.yaml`	Deployment	`frontend`	nginx con la web, 1 réplica
`31-frontend-service.yaml`	Service (NodePort)	`frontend-service`	Acceso desde fuera del clúster por el puerto 30080

4.1 Pods y Deployments

No creamos Pods directamente. Un Deployment declara el estado deseado (imagen, número de réplicas, variables, recursos y comprobaciones de salud) y crea un ReplicaSet, que es quien mantiene ese número de Pods. El backend arranca con 2 réplicas y el frontend con 1.

- **`imagePullPolicy: IfNotPresent`**: las imágenes están dentro de Minikube y no en un registro. Con `latest` Kubernetes intentaría descargarlas y los Pods acabarían en `ErrImagePull`.
- **Recursos**: cada contenedor pide CPU y memoria (`requests`, por ejemplo 100m de CPU y 128 Mi en el backend) y tiene un máximo (`limits`). El planificador usa las peticiones para elegir nodo, y el autoescalado calcula sobre ellas el porcentaje de uso.
- **Readiness probe** contra `/health`, que comprueba también la base de datos: el Service solo envía tráfico a los Pods que responden bien.
- **Liveness probe** contra `/health/live`, que no depende de la base de datos: si el proceso se cuelga, Kubernetes reinicia el contenedor. Si usara `/health`, una caída de PostgreSQL reiniciaría todos los backends sin motivo.

4.2 Base de datos: StatefulSet en lugar de Deployment

Elegimos un StatefulSet porque PostgreSQL tiene estado y un Deployment está pensado para Pods intercambiables:

- **Identidad estable**: el Pod se llama siempre `database-0` y tiene un nombre DNS propio a través del Service headless (`database-0.database-headless`). Al borrarlo, el nuevo Pod conserva el nombre.
- **Volumen propio**: `volumeClaimTemplates` crea el PersistentVolumeClaim `data-database-0`, ligado a esa identidad. Se conserva aunque el Pod se borre y el Pod nuevo vuelve a montarlo.
- **Orden**: los Pods se crean, actualizan y borran de uno en uno. Un Deployment, en una actualización, arranca el Pod nuevo antes de parar el viejo, y los dos intentarían usar el mismo directorio de datos.

Un StatefulSet de una réplica no da alta disponibilidad: subir `replicas` a 3 crearía tres bases de datos independientes, no un clúster replicado. Para eso haría falta un operador (por ejemplo CloudNativePG) o una base de datos gestionada.

Otros dos detalles del manifiesto: `PGDATA` apunta a un subdirectorio del volumen, porque en algunos entornos la raíz trae una carpeta `lost+found` y PostgreSQL no inicializa un directorio que no está vacío. Y el script `init.sql` llega desde el ConfigMap `database-init`, montado en `/docker-entrypoint-initdb.d`.

4.3 Services

Los Pods cambian de IP cada vez que se recrean, así que ningún componente usa IPs de Pods. Cada Service tiene una IP virtual fija y un nombre DNS, y selecciona sus Pods por etiqueta (`app: backend`).

Service	Tipo	Quién lo usa
`frontend-service`	NodePort (puerto 30080)	El navegador, desde fuera del clúster
`backend-service`	ClusterIP	nginx, con `BACKEND_URL=http://backend-service.smartcampus.svc.cluster.local:8000`
`database-service`	ClusterIP	El backend, con `DB_HOST=database-service`
`database-headless`	Headless (sin IP virtual)	El StatefulSet, para dar nombre DNS a cada Pod

El backend usa el nombre corto `database-service` porque Python resuelve con la configuración DNS del Pod, que añade el dominio del namespace. nginx necesita el nombre completo porque su propio *resolver* no añade dominios de búsqueda (sección 8).

4.4 ConfigMaps y Secrets

El ConfigMap `smartcampus-config` guarda la configuración no sensible. El backend y el frontend la leen como variables de entorno con `configMapKeyRef`, por ejemplo `APP_ENV`, que se ve en la web y en `/health`. El Secret `smartcampus-db-secret` guarda `DB_USER` y `DB_PASSWORD`. Los leen PostgreSQL (como `POSTGRES_USER` y `POSTGRES_PASSWORD`) y el backend, con `secretKeyRef`.

Un Secret sirve para separar las credenciales del código, de las imágenes y del resto de la configuración. Se puede dar permiso de lectura por separado con RBAC, solo se envía a los nodos donde corre un Pod que lo usa y `kubectl describe` no muestra su contenido. Frente a un gestor especializado como HashiCorp Vault o los de AWS, Azure y Google Cloud, tiene limitaciones:

- Los valores van en **base64, que no es cifrado**. En etcd se guardan sin cifrar salvo que se active el cifrado en reposo del clúster.
- Quien tenga permiso para leer Secrets o para crear Pods en el namespace puede obtener su valor.
- No hay rotación automática, ni historial de versiones, ni credenciales temporales generadas bajo demanda.

- Si el manifiesto se sube a Git, el secreto queda en el historial del repositorio.

Nosotros dejamos `02-secret.yaml` en el repositorio con una contraseña de laboratorio y un comentario que lo explica, para que el despliegue sea reproducible. En un proyecto real no se subiría: se crearía con `kubectl create secret` o se sincronizaría desde un gestor externo con herramientas como External Secrets o Sealed Secrets.

4.5 Réplicas y escalado

El backend se declara con 2 réplicas y se escala a mano con `kubectl scale deployment backend --replicas=4`. Como ampliación, `k8s/extras/hpa-backend.yaml` define un HorizontalPodAutoscaler que mantiene entre 2 y 6 réplicas para que la CPU media no pase del 50 % de lo pedido. Está en otra carpeta porque, mientras existe, deshace los cambios hechos con `kubectl scale`.

4.6 Exposición del frontend

`frontend-service` es de tipo NodePort: abre el puerto 30080 en el nodo del clúster. Con Minikube se accede con `minikube service frontend-service -n smartcampus`, que abre el navegador en la dirección correcta. El backend y la base de datos solo tienen Services ClusterIP, así que no son accesibles desde fuera. En un clúster en la nube usaríamos un Service LoadBalancer o un Ingress con dominio y certificado TLS.

5. Pruebas

Todas las pruebas del enunciado funcionaron en los dos despliegues. Las capturas son de la ejecución del 27 de octubre de 2026 en un equipo con Windows, Docker Desktop y Minikube, y los comandos exactos están en `docs/GUIA-PRUEBAS.md` del repositorio.

5.1 Docker Compose

Experimento 1 y Prueba 1: contenedores. Con `docker compose up -d --build` se construyen las imágenes y arrancan los servicios en orden. `docker compose ps` muestra los tres servicios `Up`, y el backend y la base de datos como `healthy`.

```
PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus> docker compose ps
NAME                    IMAGE                     COMMAND                  SERVICE    CREATED          STATUS                    PORTS
smartcampus-backend-1   smartcampus-backend:1.0   "uvicorn app.main:ap…"   backend    20 seconds ago   Up 8 seconds (healthy)    0.0.0.0:8000->8000/t
cp, [::]:8000->8000/tcp
smartcampus-database-1  postgres:17-alpine        "docker-entrypoint.s…"   database   21 seconds ago   Up 20 seconds (healthy)   5432/tcp
smartcampus-frontend-1  smartcampus-frontend:1.0  "/docker-entrypoint.…"   frontend   20 seconds ago   Up 1 second               0.0.0.0:8080->80/tcp
, [::]:8080->80/tcp
```

Captura 1. `docker compose ps`: los tres contenedores están en marcha, y el backend y la base de datos, `healthy`.

Prueba 2: backend. `curl http://localhost:8000/health` devuelve `{"status":"ok","service":"smartcampus-backend","environment":"development","hostname":"ab6f60b507b6","database":"ok"}`. El campo `hostname` es el identificador del contenedor que ha respondido.

```
PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus> curl.exe -s http://localhost:8000/health
{"status":"ok","service":"smartcampus-backend","environment":"development","hostname":"ab6f60b507b6","database":"ok"}
```

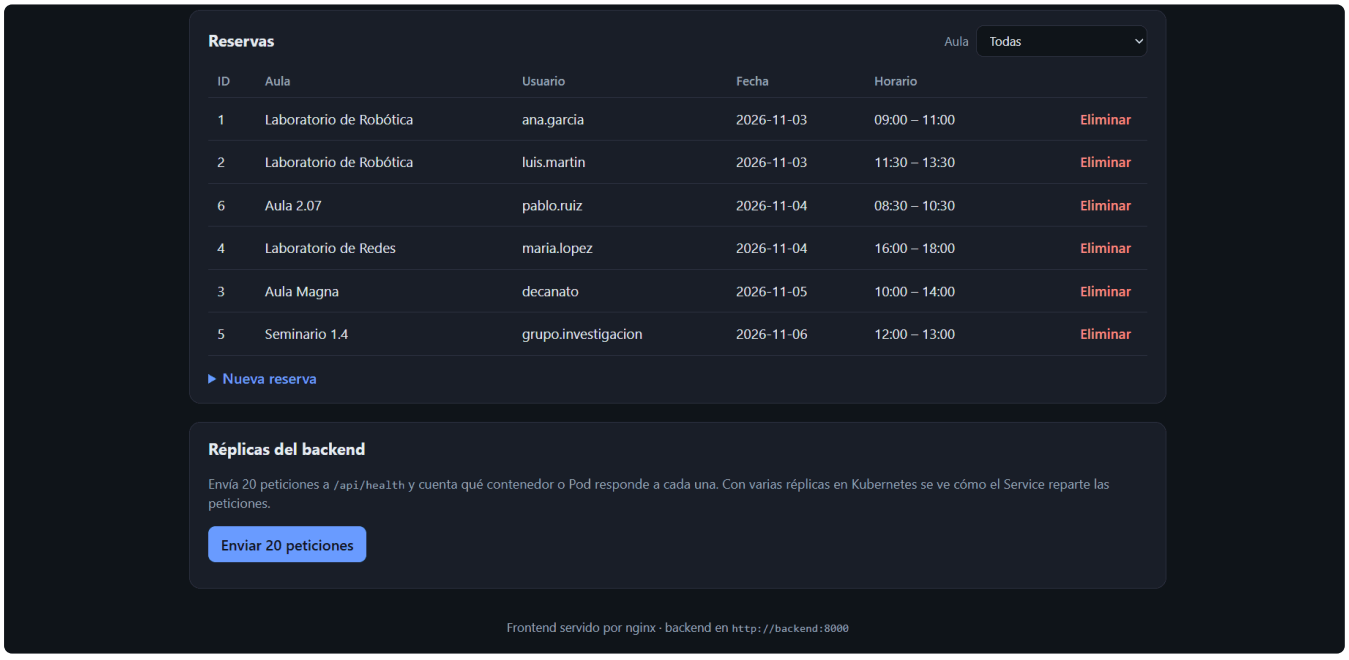
Captura 2. `/health`: estado `ok`, contenedor `ab6f60b507b6` y base de datos `ok`.

Prueba 3: comunicación con la base de datos. `/health` ejecuta `SELECT 1` en PostgreSQL y responde `"database":"ok"`, y `GET /rooms/1` devuelve el aula 1 leída de la base de datos. Desde PostgreSQL, la vista `pg_stat_activity` muestra la conexión abierta por el backend: `backend@ab6f60b507b6`, desde la IP 172.18.0.3, que es la del backend en `backend-net`. Además, `nslookup database` desde el frontend responde `NXDOMAIN`: el frontend no ve la base de datos porque no está en su red.

```
PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus> curl.exe -s http://localhost:8000/rooms/1
{"id":1,"nombre":"Laboratorio de Robótica","edificio":"Edificio A","capacidad":30,"equipamiento":["ordenadores","robots","impresora 3D"]}
PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus> docker compose exec -T database psql -U smartcampus -d smartcampus -c "SELECT application_name, client_addr, state FROM pg_stat_activity WHERE application_name LIKE 'backend%';"
   application_name    | client_addr | state
-----------------------+-------------+-------
 backend@ab6f60b507b6 | 172.18.0.3  | idle
(1 row)
```

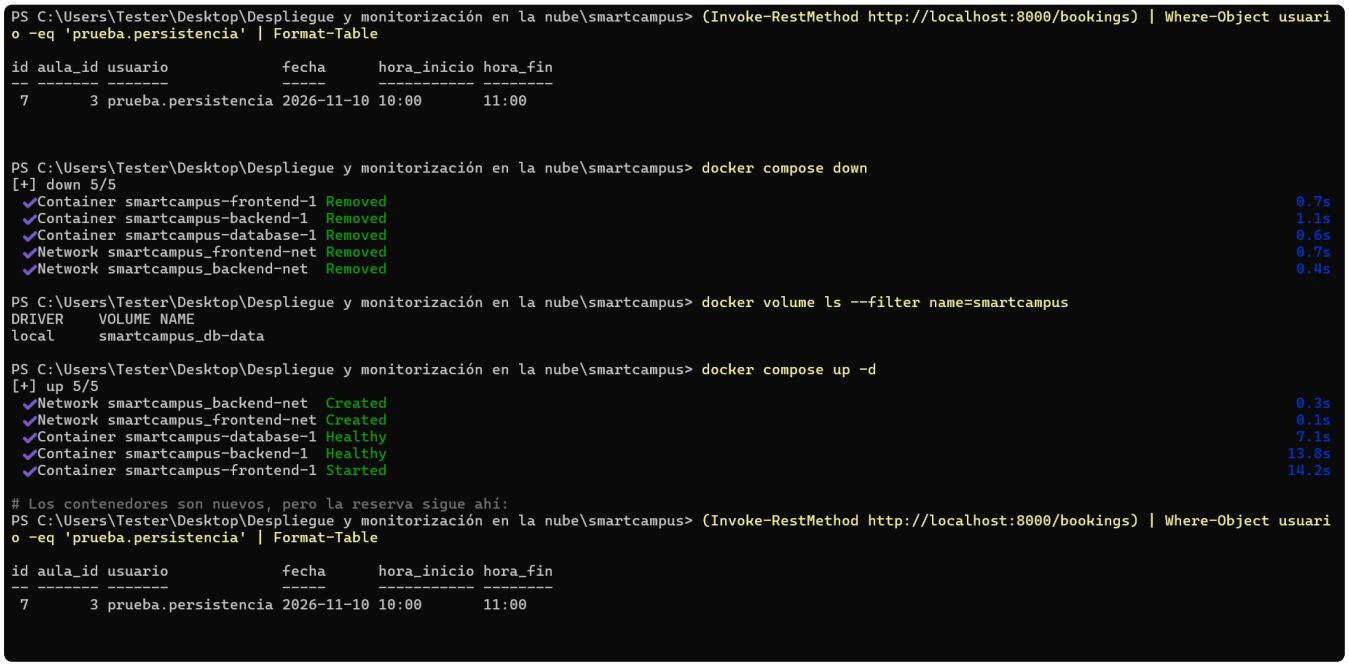
Captura 3. El backend devuelve el aula 1 (Laboratorio de Robótica) y PostgreSQL muestra su conexión, `backend@ab6f60b507b6` desde 172.18.0.3.

Prueba 4: frontend. En `http://localhost:8080` la web muestra las aulas y reservas que devuelve el backend, y el indicador verde dice qué contenedor ha respondido. Crear una reserva funciona, y una reserva solapada devuelve el error 409 "El aula ya está reservada en esa franja horaria".



Captura 4. La web en `http://localhost:8080` con las seis reservas de ejemplo que devuelve el backend. El pie indica que nginx reenvía las llamadas a `http://backend:8000`.

Prueba 5: persistencia. Creamos la reserva 7 (aula 3, usuario `prueba.persistencia`, 10 de noviembre de 2026 de 10:00 a 11:00), ejecutamos `docker compose down` (sin `-v`) y `docker compose up -d`, y la reserva seguía ahí. El volumen `smartcampus_db-data` no se borra con `down`. Con `docker compose down -v` sí se pierde, y PostgreSQL vuelve a ejecutar `init.sql`.



Captura 5. La reserva 7 antes y después de `docker compose down` y `docker compose up -d`. Los contenedores y las redes se borran, pero el volumen `smartcampus_db-data` se conserva.

Experimento 2: fallo de un contenedor.

- Con `docker stop smartcampus-backend-1` el backend queda `Exited (0)` y **no vuelve**: 30 segundos después seguía parado, `/api/health` devolvía 502 y la web mostraba "Backend no disponible". La política `restart: unless-stopped` respeta las paradas manuales.
- Con `docker compose exec backend sh -c "kill 1"` el proceso termina por sí solo. Docker vuelve a arrancar el contenedor (5 segundos después ya estaba `Up`) y `RestartCount` pasa a 1.
- Docker Compose solo recupera un servicio si se configura una política `restart` (`always`, `unless-stopped` u `on-failure`), y siempre en la misma máquina. Un healthcheck fallido marca el contenedor como `unhealthy`, pero no lo reinicia.

```
PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus> docker stop smartcampus-backend-1
smartcampus-backend-1

PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus> docker compose ps -a
NAME                    IMAGE                     COMMAND                  SERVICE    CREATED          STATUS                      PORTS
smartcampus-backend-1   smartcampus-backend:1.0   "uvicorn app.main:ap…"   backend    39 seconds ago   Exited (0) 1 second ago
smartcampus-database-1  postgres:17-alpine        "docker-entrypoint.s…"   database   39 seconds ago   Up 37 seconds (healthy)     5432/tcp
smartcampus-frontend-1  smartcampus-frontend:1.0  "/docker-entrypoint.…"   frontend   39 seconds ago   Up 24 seconds               0.0.0.0:8080->80/tcp
, [::]:8080->80/tcp

PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus> curl.exe -s -o NUL -w "HTTP %{http_code}\n" http://localhost:8080/api/he
alth
HTTP 502

# 30 segundos después...
PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus> docker compose ps -a
NAME                    IMAGE                     COMMAND                  SERVICE    CREATED              STATUS                          PORTS
smartcampus-backend-1   smartcampus-backend:1.0   "uvicorn app.main:ap…"   backend    About a minute ago   Exited (0) 36 seconds ago
smartcampus-database-1  postgres:17-alpine        "docker-entrypoint.s…"   database   About a minute ago   Up About a minute (healthy)     5432/tcp
smartcampus-frontend-1  smartcampus-frontend:1.0  "/docker-entrypoint.…"   frontend   About a minute ago   Up 59 seconds                   0.0.0.0:8080
->80/tcp, [::]:8080->80/tcp
```

```
PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus> docker compose exec -T backend sh -c "kill 1"

PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus> docker compose ps
NAME                    IMAGE                     COMMAND                  SERVICE    CREATED              STATUS                              PORTS
smartcampus-backend-1   smartcampus-backend:1.0   "uvicorn app.main:ap…"   backend    About a minute ago   Up 4 seconds (health: starting)     0.0.0.0:
8000->8000/tcp, [::]:8000->8000/tcp
smartcampus-database-1  postgres:17-alpine        "docker-entrypoint.s…"   database   About a minute ago   Up About a minute (healthy)         5432/tcp
smartcampus-frontend-1  smartcampus-frontend:1.0  "/docker-entrypoint.…"   frontend   About a minute ago   Up About a minute                   0.0.0.0:
8080->80/tcp, [::]:8080->80/tcp

PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus> docker inspect -f "{{.RestartCount}}" smartcampus-backend-1
1
```

Capturas 6 y 7. Arriba, tras `docker stop` el backend queda `Exited (0)`, el frontend responde `HTTP 502` y treinta segundos después el backend sigue parado. Abajo, tras matar el proceso principal con `kill 1`, Docker reinicia el contenedor solo (`Up 4 seconds`) y `RestartCount` vale 1.

Opcional: escalado con Compose. `docker compose up -d --scale backend=3` falla porque las tres réplicas intentan publicar el puerto 8000. Con el fichero `docker-compose.scale.yml`, que quita esa publicación, arrancan tres backends y la web reparte las peticiones entre los tres.

5.2 Kubernetes

Experimento 3: despliegue. Desde la carpeta `k8s/`, `kubectl apply -f .` crea el namespace, los dos ConfigMaps, el Secret, el StatefulSet, los dos Deployments y los cuatro Services. Los cuatro Pods (`database-0`, `backend-779c4595c7-ppgsb`, `backend-779c4595c7-vmw6m` y `frontend-74ccc6575f-b4ggm`) quedan `Running` y `1/1`, y el Deployment `backend` muestra `2/2`. La base de datos guarda sus datos en el volumen `data-database-0`, de 1 GiB, que crea la StorageClass `standard` de Minikube.

```
PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus\k8s> kubectl apply -f .
namespace/smartcampus created
configmap/smartcampus-config created
secret/smartcampus-db-secret created
configmap/database-init created
statefulset.apps/database created
service/database-service created
service/database-headless created
deployment.apps/backend created
service/backend-service created
deployment.apps/frontend created
service/frontend-service created

PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus\k8s> kubectl config set-context --current --namespace=smartcampus
Context "minikube" modified.
```

Captura 8. `kubectl apply -f .` crea los once objetos de los manifiestos, y el contexto de `kubectl` pasa a usar el namespace `smartcampus`.

```
PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus\k8s> kubectl get pods
NAME                        READY   STATUS    RESTARTS   AGE
backend-779c4595c7-ppgsb    1/1     Running   0          98s
backend-779c4595c7-vmw6m    1/1     Running   0          98s
database-0                  1/1     Running   0          99s
frontend-74ccc6575f-b4ggm   1/1     Running   0          97s
```

```
PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus\k8s> kubectl get services
NAME                TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
backend-service     ClusterIP   10.107.108.80    <none>        8000/TCP       112s
database-headless   ClusterIP   None             <none>        5432/TCP       113s
database-service    ClusterIP   10.104.235.125   <none>        5432/TCP       113s
frontend-service    NodePort    10.107.91.145    <none>        80:30080/TCP   111s
```

```
PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus\k8s> kubectl get deployments
NAME       READY   UP-TO-DATE   AVAILABLE   AGE
backend    2/2     2            2           2m5s
frontend   1/1     1            1           2m5s
```

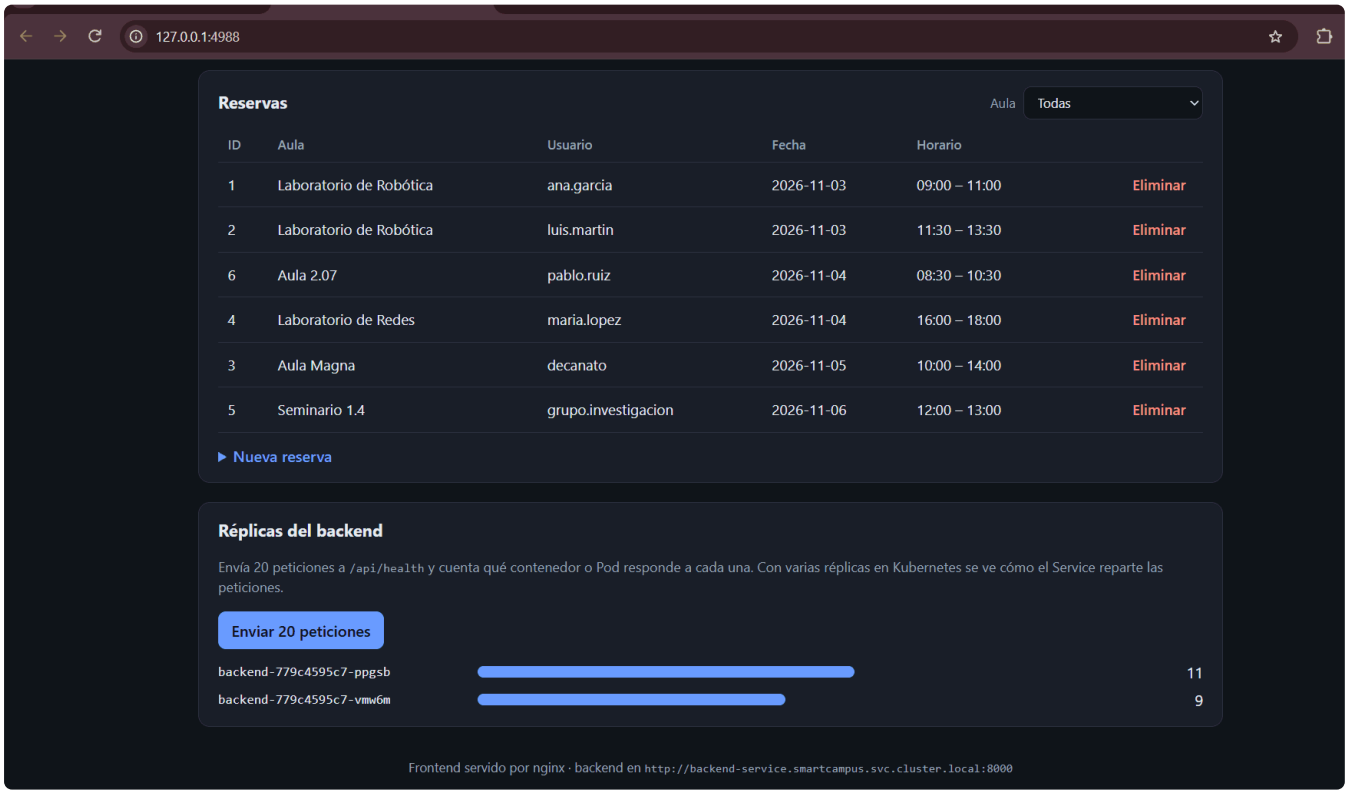
Capturas 9, 10 y 11. Los cuatro Pods en `Running` y `1/1`; los Services `backend-service` y `database-service` (ClusterIP), `database-headless` (sin IP virtual) y `frontend-service` (NodePort, puerto 30080); y los Deployments `backend` (`2/2`) y `frontend` (`1/1`).

```
PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus\k8s> kubectl get statefulsets
NAME       READY   AGE
database   1/1     2m18s

PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus\k8s> kubectl get pvc
NAME              STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
data-database-0   Bound    pvc-a0974efe-e0ee-4f7e-9b26-2cdf3d5deae0   1Gi        RWO            standard       <unset>                 2m18s
```

Captura 12. El StatefulSet `database` (1/1) y su PVC `data-database-0` (`Bound`, 1 GiB).

Exposición del frontend. `minikube service frontend-service -n smartcampus` abre la aplicación. Con el driver docker en Windows usa un túnel local, en nuestra prueba `http://127.0.0.1:4988`. La web indica "entorno: kubernetes" (valor del ConfigMap) y el nombre del Pod que responde. Con el botón "Enviar 20 peticiones", `backend-service` las repartió entre los dos Pods del backend: 11 a `ppgsb` y 9 a `vmw6m`.



Captura 13. La aplicación a través del túnel de Minikube. Las 20 peticiones se reparten entre los dos Pods (11 y 9), y el pie muestra el nombre completo del Service del backend.

Experimento 4: escalado. Tras `kubectl scale deployment backend --replicas=4`, `kubectl get pods` muestra dos Pods nuevos, `backend-779c4595c7-4q9lj` en `Pending` y `backend-779c4595c7-8ln5b` en `ContainerCreating`. Trece segundos después los dos están `Running` y el Deployment queda en `4/4`. La web reparte entonces las 20 peticiones entre los cuatro Pods (5, 4, 4 y 7). Al volver a 2 réplicas, Kubernetes pasa a `Terminating` los dos Pods más nuevos y los elimina.

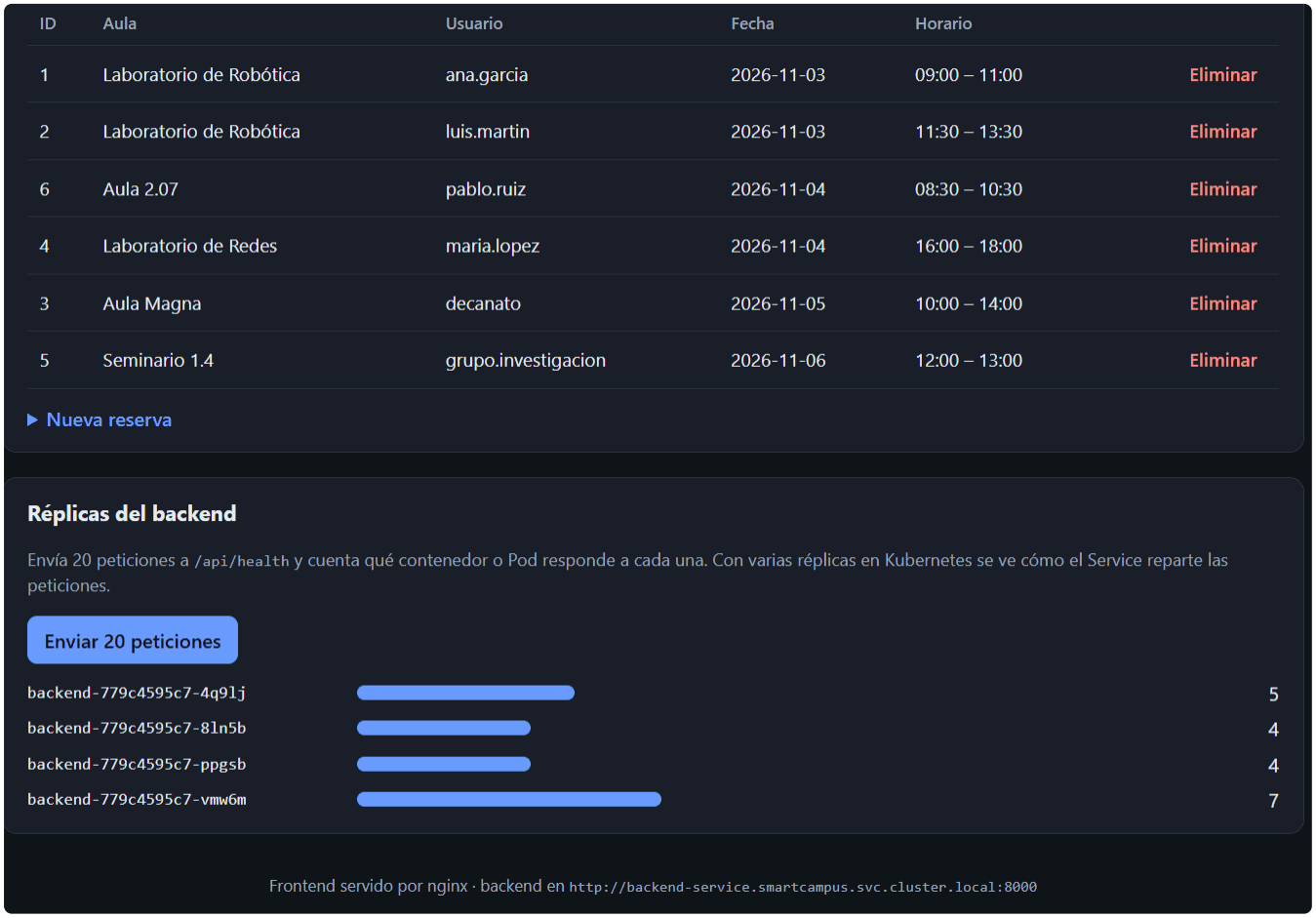
```
PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus\k8s> kubectl scale deployment backend --replicas=4
deployment.apps/backend scaled

PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus\k8s> kubectl get pods
NAME                        READY   STATUS              RESTARTS   AGE
backend-779c4595c7-4q9lj    0/1     Pending             0          0s
backend-779c4595c7-8ln5b    0/1     ContainerCreating   0          0s
backend-779c4595c7-ppgsb    1/1     Running             0          4m25s
backend-779c4595c7-vmw6m    1/1     Running             0          4m25s
database-0                  1/1     Running             0          4m26s
frontend-74ccc6575f-b4ggm   1/1     Running             0          4m24s

# unos segundos después...
PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus\k8s> kubectl get pods -l app=backend
NAME                        READY   STATUS    RESTARTS   AGE
backend-779c4595c7-4q9lj    1/1     Running   0          13s
backend-779c4595c7-8ln5b    1/1     Running   0          13s
backend-779c4595c7-ppgsb    1/1     Running   0          4m38s
backend-779c4595c7-vmw6m    1/1     Running   0          4m38s

PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus\k8s> kubectl get deployment backend
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
backend   4/4     4            4           4m39s
```

Captura 14. Al escalar a 4 réplicas aparecen dos Pods nuevos. Trece segundos después los cuatro están `Running` y el Deployment muestra `4/4`.



Captura 15. Con 4 réplicas, las 20 peticiones se reparten entre los cuatro Pods del backend: 5, 4, 4 y 7.

```
PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus\k8s> kubectl scale deployment backend --replicas=2
deployment.apps/backend scaled

PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus\k8s> kubectl get pods -l app=backend
NAME                       READY   STATUS        RESTARTS   AGE
backend-779c4595c7-4q9lj   1/1     Terminating   0          65s
backend-779c4595c7-8ln5b   1/1     Terminating   0          65s
backend-779c4595c7-ppgsb   1/1     Running       0          5m30s
backend-779c4595c7-vmw6m   1/1     Running       0          5m30s

# unos segundos después...
PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus\k8s> kubectl get pods -l app=backend
NAME                       READY   STATUS    RESTARTS   AGE
backend-779c4595c7-ppgsb   1/1     Running   0          5m33s
backend-779c4595c7-vmw6m   1/1     Running   0          5m33s
```

Captura 16. Al volver a 2 réplicas, los dos Pods creados en el escalado pasan a `Terminating` y desaparecen. Quedan `ppgsb` y `vmw6m`.

Experimento 5: self-healing.

- **Pod eliminado**: `backend-779c4595c7-ppgsb`, una de las dos réplicas del backend.
- **Qué ocurrió después**: mientras el Pod borrado pasaba a `Terminating`, apareció `backend-779c4595c7-gvjz6`. Pasó por `Pending` y `ContainerCreating`, estaba `Running` a los 2 segundos y listo (`1/1`) a los 13. Mientras tanto, la aplicación siguió respondiendo con la otra réplica, `backend-779c4595c7-vmw6m`.
- **Recurso responsable**: el ReplicaSet `backend-779c4595c7`, que gestiona el Deployment `backend`. Su controlador vio 1 Pod donde debía haber 2 y creó otro, como muestra su evento `SuccessfulCreate` para `gvjz6`.
- **Diferencia con Docker**: un contenedor parado o borrado a mano no vuelve (Experimento 2). Kubernetes no reinicia el Pod borrado: crea uno nuevo para cumplir el estado deseado, puede hacerlo en otro nodo, y el Service deja de enviar tráfico al que se elimina.

También borramos `database-0`: el StatefulSet creó otro Pod con el mismo nombre, que montó el mismo volumen, y la reserva creada antes seguía en la base de datos.

```
PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus\k8s> kubectl get pods -l app=backend
NAME                       READY   STATUS    RESTARTS   AGE
backend-779c4595c7-ppgsb   1/1     Running   0          5m59s
backend-779c4595c7-vmw6m   1/1     Running   0          5m59s

PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus\k8s> kubectl delete pod backend-779c4595c7-ppgsb
pod "backend-779c4595c7-ppgsb" deleted from smartcampus namespace

PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus\k8s> kubectl get pods -l app=backend
NAME                       READY   STATUS    RESTARTS   AGE
backend-779c4595c7-gvjz6   0/1     Running   0          2s
backend-779c4595c7-vmw6m   1/1     Running   0          6m1s
```

```
PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus\k8s> kubectl get events --sort-by=.lastTimestamp --field-selector involvedObject.kind=ReplicaSet
LAST SEEN   TYPE     REASON             OBJECT                            MESSAGE
6m19s       Normal   SuccessfulCreate   replicaset/backend-779c4595c7     Created pod: backend-779c4595c7-ppgsb
6m19s       Normal   SuccessfulCreate   replicaset/backend-779c4595c7     Created pod: backend-779c4595c7-vmw6m
6m18s       Normal   SuccessfulCreate   replicaset/frontend-74ccc6575f    Created pod: frontend-74ccc6575f-b4ggm
114s        Normal   SuccessfulCreate   replicaset/backend-779c4595c7     Created pod: backend-779c4595c7-8ln5b
114s        Normal   SuccessfulCreate   replicaset/backend-779c4595c7     Created pod: backend-779c4595c7-4q9lj
49s         Normal   SuccessfulDelete   replicaset/backend-779c4595c7     Deleted pod: backend-779c4595c7-8ln5b
49s         Normal   SuccessfulDelete   replicaset/backend-779c4595c7     Deleted pod: backend-779c4595c7-4q9lj
20s         Normal   SuccessfulCreate   replicaset/backend-779c4595c7     Created pod: backend-779c4595c7-gvjz6
```

```
kubectl get pods -w
NAME                       READY   STATUS              RESTARTS   AGE
backend-779c4595c7-ppgsb   1/1     Running             0          5m57s
backend-779c4595c7-vmw6m   1/1     Running             0          5m57s
backend-779c4595c7-ppgsb   1/1     Terminating         0          5m59s
backend-779c4595c7-ppgsb   1/1     Terminating         0          5m59s
backend-779c4595c7-gvjz6   0/1     Pending             0          0s
backend-779c4595c7-gvjz6   0/1     Pending             0          0s
backend-779c4595c7-gvjz6   0/1     ContainerCreating   0          0s
backend-779c4595c7-gvjz6   0/1     ContainerCreating   0          1s
backend-779c4595c7-ppgsb   0/1     Completed           0          6m1s
backend-779c4595c7-gvjz6   0/1     Running             0          2s
backend-779c4595c7-ppgsb   0/1     Completed           0          6m1s
backend-779c4595c7-ppgsb   0/1     Completed           0          6m1s
backend-779c4595c7-gvjz6   1/1     Running             0          13s
backend-779c4595c7-gvjz6   1/1     Running             0          13s
```

Capturas 17 y 18. Arriba, tras `kubectl delete pod backend-779c4595c7-ppgsb`, a los 2 segundos ya existe su sustituto, `backend-779c4595c7-gvjz6`. En el centro, los eventos del ReplicaSet `backend-779c4595c7`: el último `SuccessfulCreate` es `gvjz6`. Abajo, `kubectl get pods -w`: `ppgsb` pasa a `Terminating` y `gvjz6` pasa por `Pending` y `ContainerCreating` hasta quedar `Running` y `1/1`.

Experimento 6: configuración. Cambiamos `APP_ENV` de `kubernetes` a `kubernetes-v2` en `01-configmap.yaml` y lo aplicamos. El ConfigMap cambió, pero los Pods siguieron con el valor anterior: `kubectl exec deploy/backend -- printenv APP_ENV` devolvía `kubernetes`, y los Pods eran los mismos. Las variables de entorno se leen solo al arrancar el contenedor. Después de `kubectl rollout restart deployment backend frontend`, Kubernetes creó un ReplicaSet nuevo y sustituyó los Pods de uno en uno sin cortar el servicio. Los Pods nuevos, como `backend-6d8d4d7bbc-66vn6`, mostraron `kubernetes-v2` en `printenv`, en `/health` y en la web. Si el ConfigMap se montara como volumen, el fichero se actualizaría solo al cabo de un minuto aproximadamente, pero la aplicación tendría que volver a leerlo.

```
PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus\k8s> kubectl exec deploy/backend -- printenv APP_ENV
kubernetes

# Editado 01-configmap.yaml: APP_ENV pasa de "kubernetes" a "kubernetes-v2"
PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus\k8s> Select-String APP_ENV 01-configmap.yaml

01-configmap.yaml:11:  APP_ENV: "kubernetes-v2"



PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus\k8s> kubectl apply -f 01-configmap.yaml
configmap/smartcampus-config configured

PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus\k8s> kubectl get configmap smartcampus-config -o jsonpath="{.data.APP_ENV
}"
kubernetes-v2
PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus\k8s> kubectl exec deploy/backend -- printenv APP_ENV
kubernetes

PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus\k8s> kubectl get pods
NAME                       READY   STATUS    RESTARTS   AGE
backend-779c4595c7-gvjz6   1/1     Running   0          95s
backend-779c4595c7-vmw6m   1/1     Running   0          7m34s
database-0                 1/1     Running   0          7m35s
frontend-74ccc6575f-b4ggm  1/1     Running   0          7m33s
```



Capturas 19 y 20. Arriba, el fichero y el ConfigMap ya tienen `kubernetes-v2`, pero el backend sigue leyendo `kubernetes` y los Pods son los mismos de antes. Abajo, tras `kubectl rollout restart`, la web muestra "entorno: kubernetes-v2" y responde un Pod de un ReplicaSet nuevo, `backend-6d8d4d7bbc-66vn6`.

Ampliación: autoescalado. Con metrics-server activo, aplicamos el HorizontalPodAutoscaler de `k8s/extras/` (entre 2 y 6 réplicas, con un objetivo del 50 % de CPU) y lanzamos un Pod que hacía peticiones continuas a `backend-service`. El uso de CPU llegó al 219 % de lo solicitado, y el HPA subió el backend de 2 a 4 réplicas y, unos 15 segundos después, a 6, el máximo. Al quitar la carga, el HPA espera unos 5 minutos antes de reducir réplicas para no oscilar.

```
PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus\k8s> kubectl get hpa
NAME      REFERENCE            TARGETS         MINPODS   MAXPODS   REPLICAS   AGE
backend   Deployment/backend   cpu: 219%/50%   2         6         4          2m1s

PS C:\Users\Tester\Desktop\Despliegue y monitorización en la nube\smartcampus\k8s> kubectl get pods -l app=backend
NAME                       READY   STATUS    RESTARTS   AGE
backend-fffbcf955-4ng4l   0/1     Running   0          31s
backend-fffbcf955-c4tcl   0/1     Running   0          15s
backend-fffbcf955-lbhrw   1/1     Running   0          3m12s
backend-fffbcf955-m59lz   0/1     Running   0          15s
backend-fffbcf955-p7bb2   0/1     Running   0          31s
backend-fffbcf955-v88j5   1/1     Running   0          2m59s
```

Captura 21. Con la carga, el HPA mide un 219 % de CPU frente al objetivo del 50 % y el backend ya tiene 6 Pods, el máximo. La columna `REPLICAS` todavía muestra 4 porque se actualiza con retraso.

6. Comparativa Docker Compose y Kubernetes

Docker Compose arranca y conecta contenedores en una máquina; Kubernetes mantiene de forma continua un estado deseado en un clúster. Casi todas las diferencias de la tabla salen de ahí.

Característica	Docker Compose	Kubernetes
Ejecución local	Solo necesita Docker. Un `docker compose up` levanta todo en segundos y con pocos recursos.	Necesita un clúster local (Minikube, Kind o Docker Desktop), más CPU y memoria, y cargar en él las imágenes construidas.
Orquestación	Crea los contenedores en el orden de `depends_on` y ahí termina su trabajo. No reubica nada si el host falla.	Los controladores comparan sin parar el estado real con el deseado y el planificador coloca cada Pod en el nodo con recursos libres.
Escalado	Manual con `--scale`, en un solo host y en conflicto con los puertos fijos publicados (lo comprobamos en las pruebas). Sin autoescalado.	Declarativo con `replicas` o con `kubectl scale`, y automático con un HorizontalPodAutoscaler. Las réplicas se reparten entre nodos.
Self-healing	Solo la política `restart`: reinicia el mismo contenedor si su proceso termina, pero no si se para a mano ni si falla el healthcheck.	El ReplicaSet sustituye los Pods que faltan, la liveness probe reinicia contenedores colgados y la readiness probe saca del Service los que no están listos.
Service discovery	DNS interno de Docker: cada servicio se resuelve por su nombre dentro de su red.	CoreDNS da a cada Service un nombre (`<service>.<namespace>.svc.cluster.local`) y una IP virtual que no cambia aunque cambien los Pods.
Configuración	Variables en `docker-compose.yml` y en `.env`. Para cambiarlas hay que recrear el contenedor.	ConfigMaps independientes de los Pods, consumidos como variables o ficheros. Los cambios se aplican con `kubectl rollout restart`, sin corte.
Secrets	Variables de entorno o `secrets` de Compose montados como ficheros, sin control de acceso propio.	Objeto Secret con permisos RBAC propios, enviado solo a los nodos que lo usan. Sigue en base64 salvo cifrado en etcd o gestor externo.
Alta disponibilidad	No: la máquina es un punto único de fallo.	Sí, con varios nodos y réplicas: si un nodo cae, sus Pods se crean en otro. En Minikube, con un solo nodo, solo se demuestra a nivel de Pod.

Característica	Docker Compose	Kubernetes
Balanceo	El DNS devuelve las IPs de todas las réplicas y el reparto depende del cliente (en nuestro caso, nginx).	El Service reparte las conexiones entre los Pods listos. Para el tráfico externo hay NodePort, LoadBalancer e Ingress.
Actualizaciones	`docker compose up -d` recrea el contenedor, con un corte mientras arranca el nuevo. Sin historial de versiones.	Actualización progresiva: los Pods nuevos sustituyen a los viejos solo cuando están listos, y `kubectl rollout undo` vuelve a la versión anterior.
Complejidad	Un fichero y pocos conceptos: nuestro `docker-compose.yml` tiene 63 líneas sin contar comentarios.	Muchos tipos de objetos y más conocimientos para depurar: nuestros manifiestos suman 335 líneas sin comentarios en 10 ficheros.
Uso habitual	Desarrollo local, pruebas de integración, demos y aplicaciones pequeñas en un servidor.	Producción con varios servicios, varios nodos, necesidad de escalar y alta disponibilidad, normalmente en un clúster gestionado.

Kubernetes no es "mejor" en general: añade capacidades que una aplicación pequeña no necesita, a cambio de más complejidad y más recursos. Compose es la herramienta adecuada mientras una sola máquina sea suficiente y un corte breve sea aceptable.

7. Preguntas obligatorias

1. ¿Qué diferencias existen entre un contenedor Docker y un Pod de Kubernetes?

Un contenedor es un proceso aislado (con namespaces y cgroups del kernel) creado a partir de una imagen. Un Pod es la unidad mínima que gestiona Kubernetes: agrupa uno o varios contenedores que comparten IP, puertos y volúmenes, y que siempre se ejecutan juntos en el mismo nodo. Kubernetes no gestiona contenedores sueltos sino Pods, y un Pod no se repara: si desaparece, un controlador crea otro con otro nombre y otra IP. En SmartCampus cada Pod tiene un solo contenedor, pero se podría añadir otro auxiliar en el mismo Pod, por ejemplo un agente de logs.

2. ¿Qué problema resuelve Docker Compose?

Permite describir en un solo fichero una aplicación de varios contenedores (servicios, imágenes, redes, volúmenes, variables y dependencias) y levantarla o pararla con un comando. Sin Compose habría que crear redes y volúmenes y lanzar cada `docker run` a mano, con todas sus opciones y en el orden correcto. Además, el fichero hace el entorno reproducible para todo el equipo.

3. ¿Qué problemas adicionales intenta resolver Kubernetes?

Los de ejecutar contenedores en producción y en varias máquinas: decidir en qué nodo corre cada Pod, mantener el número de réplicas aunque fallen Pods o nodos, ofrecer nombres y balanceo estables, escalar a mano o automáticamente, actualizar sin cortes con vuelta atrás, gestionar configuración y secretos, y desacoplar el almacenamiento persistente de los Pods. Todo ello con una API declarativa: se describe el estado deseado y el clúster trabaja para mantenerlo.

4. ¿Qué ocurre si eliminamos manualmente un contenedor ejecutado mediante Docker Compose?

El servicio deja de funcionar y nadie lo vuelve a crear. La política `restart` respeta las paradas manuales: en el Experimento 2 el backend siguió parado tras `docker stop` y el frontend devolvía 502. Hay que volver a ejecutar `docker compose up -d` o `docker compose start`. Solo si el proceso termina por sí solo, `restart: unless-stopped` o `always` reinician el contenedor.

5. ¿Qué ocurre si eliminamos un Pod gestionado por un Deployment?

El ReplicaSet que creó el Deployment detecta que hay menos Pods de los deseados y crea uno nuevo en unos segundos, con otro nombre y otra IP. Mientras tanto, el Service deja de enviar tráfico al Pod eliminado y lo reparte entre las réplicas que quedan, así que la aplicación sigue respondiendo (Experimento 5).

6. ¿Cómo realiza Kubernetes el descubrimiento de servicios?

Cada Service recibe una IP virtual estable y un nombre DNS en el DNS del clúster (CoreDNS): `<service>.<namespace>.svc.cluster.local`. Los Pods tienen ese DNS configurado con dominios de búsqueda, así que dentro del mismo namespace basta el nombre corto, como hace el backend con `database-service`. El Service elige sus Pods por etiquetas y Kubernetes mantiene actualizada la lista de sus IPs (EndpointSlices). Kubernetes también inyecta variables de entorno con la dirección de cada Service que ya existe cuando arranca el Pod.

7. ¿Por qué no es recomendable utilizar directamente la IP de un Pod?

Porque cambia cada vez que el Pod se recrea, se escala o se mueve de nodo: en el Experimento 5 el Pod nuevo tenía otra IP. Además, una IP apunta a una sola réplica, así que no hay balanceo y no se excluyen los Pods que no están listos. El Service resuelve los dos problemas con una IP y un nombre estables.

8. ¿Qué función cumple un Deployment?

Declara el estado deseado de una aplicación sin estado: imagen, número de réplicas, variables, comprobaciones de salud y recursos. Crea un ReplicaSet que mantiene ese número de Pods. Cuando cambia la plantilla, crea un ReplicaSet nuevo y sustituye los Pods de forma progresiva, y guarda el historial para volver atrás con `kubectl rollout undo`.

9. ¿Qué función cumple un Service?

Da un punto de acceso estable (IP virtual y nombre DNS) a un grupo cambiante de Pods, seleccionados por etiquetas, y reparte el tráfico entre los que están listos. Según el tipo, el acceso es solo interno (ClusterIP), por un puerto de cada nodo (NodePort) o mediante un balanceador externo (LoadBalancer). Un Service headless no tiene IP virtual y devuelve las IPs de los Pods.

10. ¿Qué diferencias existen entre un Deployment y un StatefulSet?

	Deployment	StatefulSet
Nombres de los Pods	Aleatorios (`backend-779c4595c7-ppgsb`)	Fijos y ordenados (`database-0`, `database-1`)
Red	Solo a través del Service	Nombre DNS propio por Pod mediante un Service headless
Almacenamiento	Todos los Pods comparten el mismo volumen, si lo hay	Un PersistentVolumeClaim por réplica, que se conserva al recrear el Pod
Creación y actualización	En paralelo; en una actualización arranca el Pod nuevo antes de parar el viejo	En orden, de uno en uno
Uso típico	Aplicaciones sin estado, como el backend	Bases de datos y sistemas con estado

11. ¿Por qué una base de datos puede requerir un tratamiento diferente al de una API stateless?

Una API sin estado puede perder cualquier réplica sin perder datos, porque los datos están en la base de datos. La base de datos es el estado: necesita almacenamiento que sobreviva al Pod, que nunca lo usen dos instancias a la vez (se corrompería), una identidad estable y paradas ordenadas. Tampoco escala solo con más réplicas: varias instancias necesitan replicación entre una primaria y sus réplicas, que gestiona un operador o un servicio gestionado.

12. ¿Qué problema solucionan ConfigMap y Secret?

Separan la configuración del código y de la imagen, de modo que la misma imagen sirve para desarrollo, pruebas y producción cambiando solo esos objetos. El ConfigMap guarda datos no sensibles (`APP_ENV` , `BACKEND_URL`) y el Secret, credenciales (`DB_USER` , `DB_PASSWORD`), con permisos de acceso separados. Los dos se entregan a los Pods como variables de entorno o como ficheros.

13. ¿Cómo se escala la aplicación con Docker Compose?

Con `docker compose up -d --scale backend=3` o con `deploy.replicas` en el fichero. Tiene límites: todas las réplicas están en el mismo host, falla si el servicio publica un puerto fijo (nos pasó con el 8000 y usamos `docker-compose.scale.yml`), el reparto depende del cliente y no hay autoescalado. En nuestro caso funciona porque el DNS de Docker devuelve las IPs de las tres réplicas y nginx las alterna.

14. ¿Cómo se escala la aplicación con Kubernetes?

Cambiando `replicas` en el Deployment y aplicándolo, o con `kubectl scale deployment backend --replicas=4` . Kubernetes coloca los Pods nuevos en nodos con recursos libres y el Service les envía tráfico cuando pasan la readiness probe. Con un HorizontalPodAutoscaler el escalado es automático según la CPU u otras métricas: en nuestra prueba subió el backend de 2 a 6 réplicas bajo carga.

15. ¿Qué sucede cuando un Pod falla?

Depende del fallo:

- Si el contenedor termina o falla la liveness probe, el kubelet lo reinicia dentro del mismo Pod, esperando cada vez más entre intentos (`CrashLoopBackOff` si se repite).
- Si falla la readiness probe, el Pod sigue vivo pero sale del Service hasta que se recupera.

- Si el Pod desaparece porque se borra, se desaloja o cae su nodo, el ReplicaSet crea otro para mantener el número de réplicas.

16. ¿Qué ventajas aporta Kubernetes para una aplicación formada por múltiples servicios?

Un modelo declarativo común para todos los servicios, descubrimiento por DNS y balanceo integrados, y escalado independiente por servicio (por ejemplo, 4 backends y 1 frontend). Añade recuperación automática, despliegues sin corte con vuelta atrás, configuración y secretos centralizados, aislamiento con namespaces, límites de recursos y permisos, y portabilidad entre proveedores cloud.

17. ¿Qué inconvenientes introduce Kubernetes frente a Docker Compose?

Más complejidad: muchos conceptos y objetos, más YAML (10 ficheros frente a 1 en nuestro caso) y una depuración que exige entender la red y los controladores. Consume más recursos, en local obliga a cargar las imágenes en el clúster, y en producción hay que operar o pagar el clúster. Para una aplicación pequeña, ese coste no compensa.

18. ¿En qué escenario utilizaríais Docker Compose?

En desarrollo local y demos, en pruebas de integración que levantan la aplicación completa en cada ejecución de CI, y en aplicaciones pequeñas o internas que caben en un servidor y toleran un corte breve.

19. ¿En qué escenario utilizaríais Kubernetes?

En producción con varios servicios o equipos, cuando hacen falta alta disponibilidad, escalado y despliegues sin corte, cuando la carga varía a lo largo del día, y cuando se quiere la misma forma de desplegar en varios entornos o proveedores.

20. ¿Qué cambios serían necesarios para llevar esta aplicación desde un entorno local hasta un entorno cloud?

- **Imágenes**: publicarlas en un registro (Docker Hub, Amazon ECR, Google Artifact Registry o Azure Container Registry) con etiquetas de versión, en lugar de cargarlas en Minikube.
- **Clúster**: uno gestionado (EKS, GKE o AKS) con varios nodos en distintas zonas.
- **Exposición**: un Ingress o un Service LoadBalancer con dominio propio y TLS, en lugar de NodePort.
- **Base de datos**: un servicio gestionado (Amazon RDS, Cloud SQL, Azure Database for PostgreSQL) o un operador con réplicas y copias de seguridad.

- **Secretos**: en un gestor externo sincronizado con el clúster y cifrado de etcd, fuera del repositorio.
- **Disponibilidad y escalado**: varias réplicas del frontend, HPA en los servicios con carga variable y PodDisruptionBudgets.
- **Seguridad**: NetworkPolicies para limitar quién habla con la base de datos (el equivalente a las dos redes de Compose), RBAC y escaneo de imágenes.
- **Automatización**: un pipeline de CI/CD que construya, pruebe y despliegue, con Helm, Kustomize o GitOps.
- **Monitorización**: métricas, logs centralizados y alertas, por ejemplo con Prometheus, Grafana y Loki o con el servicio del proveedor.

8. Problemas encontrados

Durante el desarrollo y las pruebas aparecieron siete problemas de infraestructura. La tabla recoge para cada uno el síntoma, la causa que diagnosticamos y la solución aplicada.

#	Fase	Síntoma	Causa	Solución
1	Construcción de imágenes	`pip install` fallaba dentro de `docker build` con `CERTIFICATE_VERIFY_FAILED: self-signed certificate in certificate chain`.	La red salía a Internet por un proxy que inspecciona el tráfico TLS con su propia autoridad de certificación. El equipo confiaba en ella, pero el contenedor de construcción no.	Añadir el certificado del proxy a la imagen base solo en esa red (`PIP_CERT` y `SSL_CERT_FILE`), sin tocar el Dockerfile del proyecto. Fuera de esa red no hace falta.
2	Comunicación entre servicios (Kubernetes)	Con `BACKEND_URL=http://backend-service:8000` , la web devolvía `502 Bad Gateway` en todas las llamadas a `/api`.	nginx resuelve el nombre con su propio *resolver* en cada petición, y ese resolver no añade los dominios de búsqueda de `/etc/resolv.conf`. El nombre corto no existe para él.	Poner el nombre completo `backend-service.smartcampus.svc.cluster.local` en el ConfigMap. En Compose el nombre corto sí funciona porque el DNS de Docker lo resuelve directamente.
3	Despliegue con Compose	`docker compose up -d --scale backend=3` fallaba con `Bind for 0.0.0.0:8000 failed: port is already allocated`.	Las tres réplicas intentaban publicar el mismo puerto 8000 del equipo.	El fichero `docker-compose.scale.yml` quita la publicación del puerto (`ports: !reset []`). El frontend sigue llegando al backend por la red interna.
4	Despliegue con Compose (Experimento 2)	`docker exec smartcampus-backend-1 kill 1` respondía `exec: "kill": executable file not found in $PATH`.	La imagen `python:3.12-slim` no incluye el programa `kill` (paquete procps).	Usar el `kill` integrado en la shell: `docker compose exec backend sh -c "kill 1"`.

#	Fase	Síntoma	Causa	Solución
5	Kubernetes (clúster local)	El clúster no arrancaba: kubeadm esperaba al kubelet hasta agotar el tiempo, con el aviso de que cgroups v1 ya no está soportado.	El sistema usaba cgroups v1, y desde Kubernetes 1.35 el kubelet no arranca con cgroups v1 salvo que se configure expresamente.	Crear el clúster con Kubernetes 1.34 (por ejemplo, `minikube start --kubernetes-version=v1.34.3`). La alternativa es pasar el sistema a cgroups v2.
6	Kubernetes (manifiestos)	Tras probar un cambio con `kubectl set env deployment/frontend BACKEND_URL=...`, `kubectl apply -f 30-frontend-deployment.yaml` fallaba con `valueFrom: Invalid value: "": may not be specified when value is not empty`.	`kubectl set env` cambió la variable de `valueFrom` (ConfigMap) a `value` (texto), y `apply` combinó los dos campos en el mismo objeto.	Restaurar el Deployment con `kubectl replace -f 30-frontend-deployment.yaml` y hacer los cambios siempre en los ficheros YAML.
7	Pruebas en Windows (Docker Compose)	Al lanzar `docker compose up -d --build` desde el script de PowerShell que automatiza las capturas, se descargaba la imagen de PostgreSQL, pero el comando terminaba con `failed to get console: The handle is invalid` (código de salida 1).	El script pasaba la salida de Docker por una tubería de PowerShell (`| Out-Host`). En Windows, Compose usa la consola para dibujar el progreso y, con la salida redirigida, no encuentra una consola válida.	Lanzar `docker` desde el script sin redirigir su salida (con `Process.Start`). Escrito a mano en la terminal, el comando funciona sin cambios.

9. Conclusiones

Usaríamos Docker Compose mientras la aplicación quepa en una máquina, y Kubernetes cuando hagan falta varias máquinas, alta disponibilidad o escalado. Las dos herramientas usan las mismas imágenes, así que pasar de una a otra no obliga a cambiar el código.

Escenario	Tecnología	Justificación
Desarrollo local	Docker Compose	Levanta los tres servicios en segundos con un comando y pocos recursos. Los logs y la depuración son directos.
Pruebas	Docker Compose, y un clúster local efímero para probar los manifiestos	En integración continua se crea y destruye el entorno completo en cada ejecución. Kind o Minikube validan los manifiestos antes de producción.

Escenario	Tecnología	Justificación
Pequeño proyecto personal	Docker Compose en un solo servidor	Operar un clúster cuesta más de lo que aporta. `restart: unless-stopped` y copias del volumen cubren lo necesario.
Aplicación empresarial con múltiples servicios	Kubernetes, preferiblemente gestionado (EKS, GKE o AKS)	Descubrimiento de servicios, despliegues sin corte, permisos por equipo con namespaces y RBAC, y alta disponibilidad entre nodos.
Aplicación que necesita escalar	Kubernetes con HorizontalPodAutoscaler	Escala cada servicio por separado y de forma automática, como comprobamos con el backend al pasar de 2 a 6 réplicas bajo carga.

La diferencia de fondo es quién mantiene el estado deseado. Con Compose lo mantiene la persona que ejecuta los comandos: si un contenedor se para, alguien tiene que volver a arrancarlo. Con Kubernetes lo mantienen los controladores del clúster, y por eso el escalado y el self-healing funcionaron sin intervención. Un flujo razonable para SmartCampus es desarrollar con Compose y desplegar en Kubernetes.